P1877R0
Saving Private Ranges: Recovering Lost Information from Comparison and Predicate Algorithms

Published Proposal, 2019-10-07

This version:
https://thephd.github.io/vendor/future_cxx/papers/d1803.html

Author:
JeanHeyd Meneide

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:
C++20

Latest:
https://thephd.github.io/vendor/future_cxx/papers/d1803.html

Abstract

Currently, many of the new algorithms in std::ranges algorithms copy their `bool`-returning predecessors by returning a single `bool` value. And while this makes perfect sense, developers who build algorithms on top of the standard ones often have to replicate the information that these algorithms already perform. This proposal adjusts the return types of std::ranges algorithms to return more information and be less lossy, preventing people from redoing work already performed by the called algorithm and without having to replicate the internal metaprogramming and state information that implementations already come across as a natural consequence of the algorithm.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 0 - October 7th, 2019

— Initial release.

§ 2. Motivation

When building wrapping ranges and iterators, it is useful for individuals working with these wrapped iterators to provide algorithms synonymous to the C++ Standard. For example, if someone is writing a `bit_iterator` or a `code_point_iterator` which wraps an underlying iterator and performs operations on the underlying iterator, one can optimize for the case where the wrapped iterators are potentially of the same `iterator_category` (`iterator_concept`) and have the same `value_type`s. In an example from real-world code:

```
template<typename _It0, typename _It1>
constexpr bool
bit_equal(bit_iterator<_It0> __first0, bit_iterator<_It0> __last0,
        bit_iterator<_It1> __first1)
{
        using __iterator0        = __bit_iterator<_It0>;
        using __iterator1        = __bit_iterator<_It1>;
        using __difference_type0 = typename ::std::iterator_traits<__iterat
        using __iterator_category0 = typename __iterator0::iterator_categor
        using __iterator_category1 = typename __iterator1::iterator_categor
        using __base_iterator0     = typename __iterator0::iterator_type;
        using __base_iterator1     = typename __iterator1::iterator_type;
        using __base_value_type0 = typename ::std::iterator_traits<__base_i
        using __base_value_type1 = typename ::std::iterator_traits<__base_i
        if constexpr (::std::is_unsigned_v<__base_value_type0> &&
                                ::std::is_unsigned_v<__base_value_type1> &&
                                ::std::is_same_v<__base_value_type0, __base
        {
                if constexpr (__is_iterator_category_or_better_v<::std::for
                                __iterator_category0>
                        && __is_iterator_category_or_better_v<::std::forwar
                                __iterator_category1>)
                        {
                                // get .base() and use internal algorithms.
                        }
        }
        // use baseline input algorithm...
}
```

In the innermost branch after checking iterator categories, we would like to use `std::equal` on the `.base()` iterators, to compare whole words at a time or entire sequences at a time rather than just compare 1 bit at a time. The problem with using `std::equal` here is that it only returns a `bool` value: if there is any additional "work" left over after `std::equal` is done comparing the fully populated underlying iterators, we now have to manually re-increment all the way to the end.

This problem is present with a large number of algorithms in the standard. From `copy`/`copy_n` to `equal`, many algorithms advance the iterator or perform useful computation on the iterators that is then discarded, leaving higher levels to re-do that work and incur a performance penalty (violating the idea that it could not be done by hand better with optimizations turned on).

This performance penalty was fixed for certain `std::ranges` algorithms. For example, `copy`, where a `std::ranges::copy` returns a `std::ranges::copy_result<Iterator, OutputIterator>`, allows someone to retrieve the underlying incremented `Iterator` type.

There are three ways around the problem:

1. re-implement what the `std::ranges` algorithms do now, which is what libstdc++ has done for a handful of algorithms before `std::ranges` came along. This works only for an individual library's implementation of that algorithm;
2. re-implement any of the time-complexity checks and then use a "lower level" algorithm to perform the rest of the work. For example, this would include duplicating logic from `std::ranges::equal` to check sizes in the case of random access. After doing explicit `std::distance` checks upon getting `random_access_iterator`s or better to meet the complexity requirements of `std::equal`, a developer would then dispatch to its "lower level" algorithmic form by calling `std::ranges::mismatch`, which does return iterator information;
3. or, return iterator information from the `std::ranges` version of the algorithms that currently lose this information by returning only a `bool`.

This paper proposes Option 3, which is enhancing only the `std::ranges` versions of these algorithms to return additional iterator information, in the same way `std::ranges::copy` was enhanced over its non-`ranges` counterpart. This is a backwards-compatible change since it does not touch the original algorithms.

§ 3. Design

The design here is fairly straightforward: we go through all algorithms returning `bool` in the standard library's `std::ranges` namespace and change it to have a return type similar to the below structure. This proposal uses the same machinery that other range algorithms like `std::ranges::mismatch` and friends to produce an `X_result` type. Most algorithms only need a `predicate_result` type, while others require a little extra information.

§ 3.1. Single-Boolean Predicate Returns

The following result structure...

```
namespace ranges {
        template<class I1>
        struct predicate_result {
                [[no_unique_address]] I1 in;
                bool value;

                template<class II1>
                        requires convertible_to<const I1&, II1>
                        operator predicate_result<II1>() const & {
                                return {in, value};
                        }

                template<class II1>
                        requires convertible_to<I1> && convertible_to<I2>
                        operator predicate_result<II1>() && {
                                return {std::move(in), value};
                        }

                explicit operator bool () const {
                        return value;
                }
        };
}
```

... works for the following algorithm return types.

— In Non-modifying Algorithms [alg.nonmodifying]:

— `std::ranges::all_of`;

— `std::ranges::any_of`;

— and, `std::ranges::none_of`.

— In Sorting and related operations [alg.sorting]:

— `std::ranges::is_sorted`;

— `std::ranges::is_partitioned`;

— and, `std::ranges::is_heap`.

It is notable that `is_sorted`, `is_partitioned`, and `is_heap` all have versions of themselves that return an iterator and also do not have additional complexity requirements specified in the standard

over their `is_{}_until` versions. Therefore, it may be prudent to just leave changes to these algorithms off entirely rather than also return both a convenience `bool` value and the iterator.

Likewise, `any_of`, `all_of`, and `none_of` can be seen as wrappers around the `find_if(_not)` algorithms. None of them impose additional complexity requirements, nor do standard libraries today do anything particularly special in the general implementation of these wrappers either. While the information could be returned, simply using a lower-level facility would be suitable in the cases here.

§ 3.2. Single-Boolean Comparison Returns

Similarly, the following `comparison_result` structure...

```
namespace ranges {
        template<class I1, class I2>
        struct comparison_result {
                [[no_unique_address]] I1 in1;
                [[no_unique_address]] I2 in2;
                bool value;

                template<class II1, class II2>
                        requires convertible_to<const I1&, II1> && converti
                        operator comparison_result<II1, II2>() const & {
                                return {in1, in2, value};
                        }

                template<class II1, class II2>
                        requires convertible_to<I1, II1> && convertible_to<
                        operator predicate_result<II1, II2>() && {
                                return {std::move(in1), std::move(in2), val
                        }

                explicit operator bool () const {
                        return value;
                }
        };
}
```

... works for algorithms which take 2 ranges.

— In Non-modifying Algorithms [alg.nonmodifying]:

— `std::ranges::equal`;

— and, `std::ranges::is_permutation`.

— In Sorting and related operations [alg.sorting]:

— `std::ranges::binary_search`;

— and, `std::ranges::lexicographic_compare`.

Algorithms such as `std::ranges::includes` are not included in this fix up because that algorithm works with input ranges, and the iterators returned from it by the time the algorithm completes can essentially be empty husks that contain no valuable information. The goal would be to have a well-specified return value for the iterators on algorithm success, where logically that information implies the whole range was examined.

`std::ranges::lexicographic_compare` returns whether or not the range is in lexicographic order: an enhancement to that would be to return the incremented-to-the-end `first1` and `first2` iterators in the case where the algorithm returns `true` for the result. Otherwise, the value of `in1` and `in2` are unspecified. Similarly for `std::ranges::equal`: the incremented iterators should be returned from `equal` in `in1` and `in2` if the algorithm returns true: otherwise, the value of the returned iterators is unspecified.

§ 3.3. Questions

Question 1: These algorithms take in a range. We recognize that the result type is likely more ergonomic and efficient in terms of the amount of times an iterator has to be moved to store the result, and so recommended return types that return iterators like the other modified `std::ranges` algorithms. Still,

> is it worthwhile to have the return results return a `std::ranges::subrange<...>` type, rather than just a `result` type with the iterators?

Question 2: Some of these algorithms are more or less trivial wrappers around their counterparts with no additional complexity requirements or guarantees obtain by metaprogramming checks. These are `std::ranges::is_sorted`, `std::ranges::is_partitioned`, and `std::ranges::is_heap`.

> Is returning the iterator worth it when there are `X_until` versions of all these algorithms which do return the iterator and can be defined as convenience wrappers?